REST APIs

Slides by Raed Almomani & Andrew DeOrio

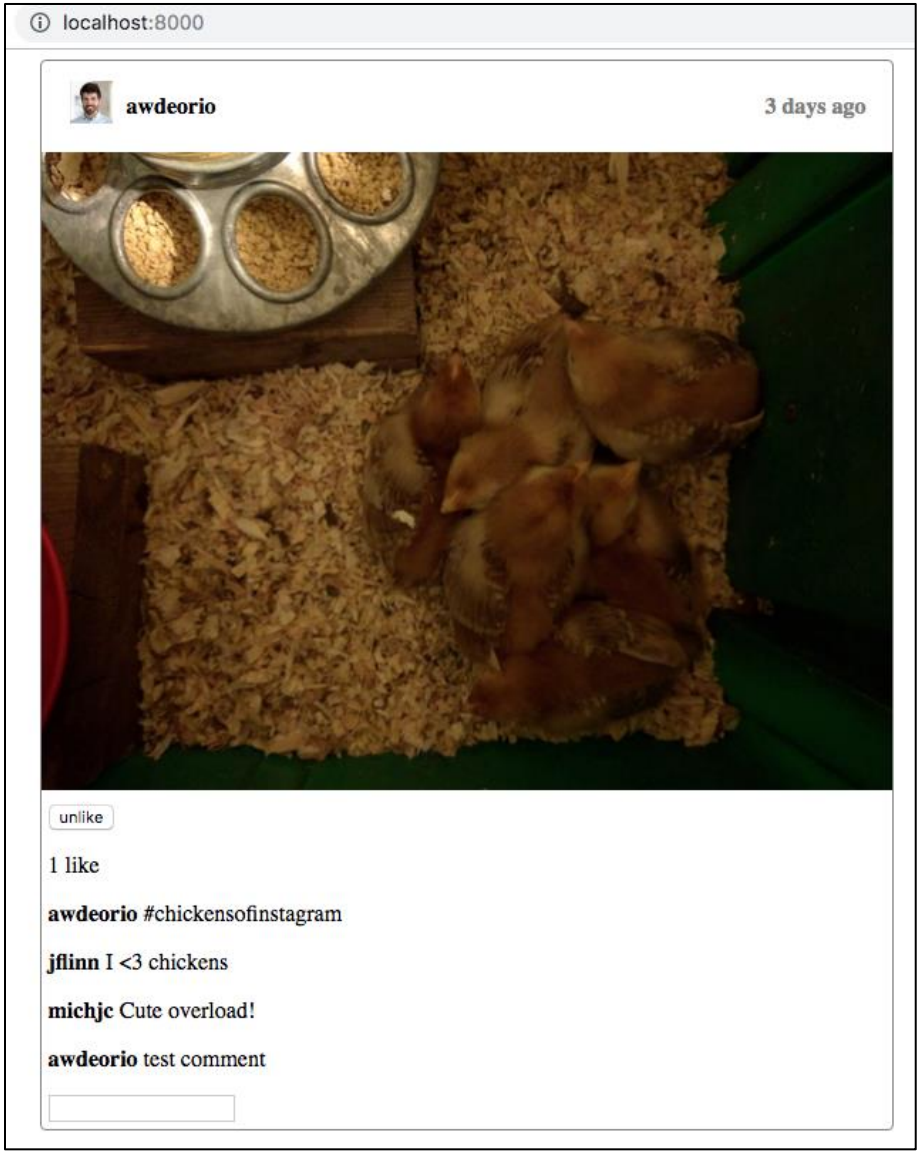
Agenda

- **REST API intro**
- JSON
- Collections and pagination
- REST API Verbs and Status codes.
- Safety and idempotence

REST API

- REST
  - "Representational State Transfer"
- API
  - "Application Programming Interface"
- How Client/Server share data.

Not a REST API



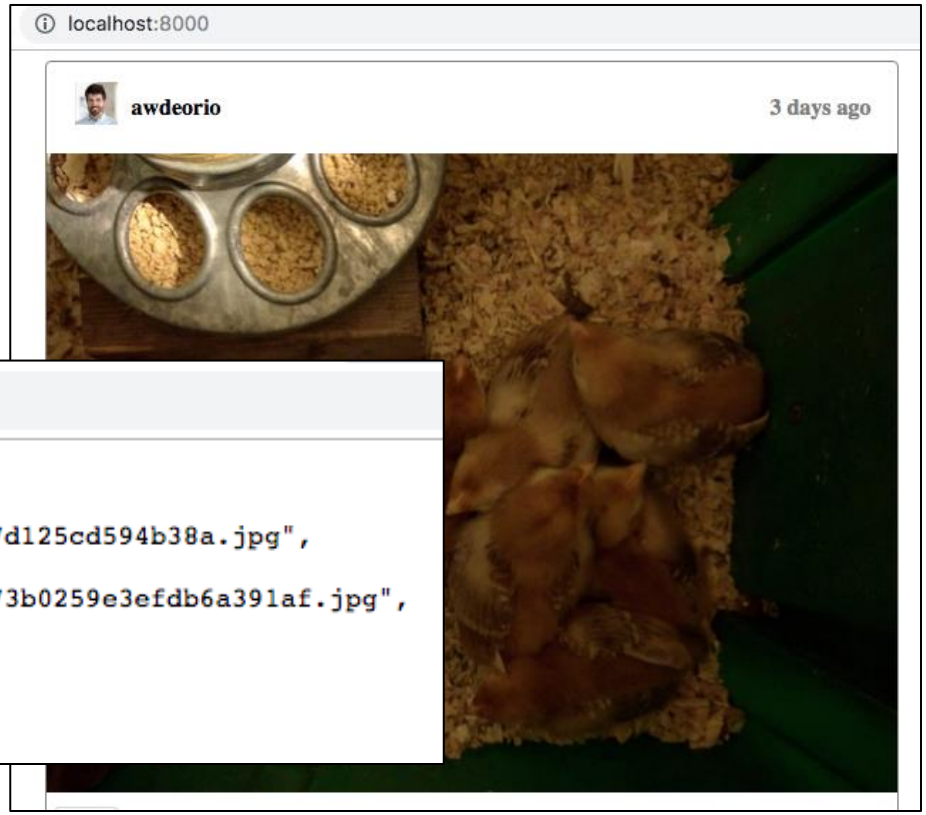
REST API

localhost:8000/api/v1/p/3/


```
{
  "age": "2019-09-20 17:28:59",
  "img_url": "/uploads/9887e06812ef434d291e4936417d125cd594b38a.jpg",
  "owner": "awdeorio",
  "owner_img_url": "/uploads/e1a7c5c32973862ee15173b0259e3efdb6a391af.jpg",
  "owner_show_url": "/u/awdeorio/",
  "post_show_url": "/p/3/",
  "url": "/api/v1/p/3/"
}
```

REST API

- JavaScript runs in browser
- JavaScript receives JSON data from server
- JavaScript renders data on page



localhost:8000/api/v1/p/3/


```
{
  "age": "2019-09-20 17:28:59",
  "img_url": "/uploads/9887e06812ef434d291e4936417d125cd594b38a.jpg",
  "owner": "awdeorio",
  "owner_img_url": "/uploads/e1a7c5c32973862ee15173b0259e3efdb6a391af.jpg",
  "owner_show_url": "/u/awdeorio/",
  "post_show_url": "/p/3/",
  "url": "/api/v1/p/3/"
}
```

REST APIs use HTTP

- HTTP request includes a Method
- HTTP response includes a Status code and JSON data

```
➢GET /api/v1/p/1/ HTTP/1.0

< HTTP/1.0 200 OK
< Content-Type: application/json
{
  "age": "2019-09-20 17:28:59",
  "img_url": "/uploads/122a7d27ca1d7420a1072f695d9290fad4501a41.jpg",
  "owner": "awdeorio",
  "owner_img_url": "/uploads/e1a7c5c32973862ee15173b0259e3efdb6a391af.jpg",
  ...
}
```

Agenda

- REST API intro
- **JSON**
- Collections and pagination
- REST API Verbs and Status codes.
- Safety and idempotence

JSON

- JSON: JavaScript Object Notation.
- Lightweight data-interchange format
- Based on JavaScript syntax
- Commonly used to send data from a server to a web client

JSON structures

- Object: a collection of name/value pairs

```
{ "name": "DeOrio", "num_chickens": 4 }
```

- Array: an ordered list of values

```
[ "Marilyn", "Maude", "Myrtle II", "Mabel"]
```

- A value is:
  - string
  - number
  - `true`
  - `false`
  - `null`
  - Object
  - Array

Valid JSON

- Pitfall: **no trailing commas allowed!**

```
{
  "login":"awdeorio",
  "id":7503005,
  ...


   "updated_at": "2017-12-12T19:11:17Z” /

}
```

- More details: http://www.json.org/

Example JSON for Insta485 image post

localhost:8000/api/v1/p/3/


```
{
  "age": "2019-09-20 17:28:59",
  "img_url": "/uploads/9887e06812ef434d291e4936417d125cd594b38a.jpg",
  "owner": "awdeorio",
  "owner_img_url": "/uploads/e1a7c5c32973862ee15173b0259e3efdb6a391af.jpg",
  "owner_show_url": "/u/awdeorio/",
  "post_show_url": "/p/3/",
  "url": "/api/v1/p/3/"
}
```

Agenda

- REST API intro
- JSON
- **Collections and pagination**
- REST API Verbs and Status codes.
- Safety and idempotence

Detail view AKA item view

- Our previous example showed a REST API *detail view* or *item view*
- It returns one object from the database
- Notice the post id part of the URL
  - Also called a "slug"

```
$ GET /api/v1/posts/1/ HTTP/1.0
{
  "age": "2019-09-20 17:28:59",
  "img_url": "/uploads/122a7d27ca1d7420a1072f695d9290fad4501a41.jpg",
  "owner": "awdeorio",
  "owner_img_url": "/uploads/e1a7c5c32973862ee15173b0259e3efdb6a391af.jpg",
  ...
  "url": "/api/v1/posts/1/"
}
```

List view AKA collection view

- REST APIs often expose collections of items

```
$ GET /api/v1/posts/ HTTP/1.0
```

```
{
  "results": [
    {
      "postid": 3,
      "url": "/api/v1/posts/3/"
      ...
    },
    {
      "postid": 2,
      "url": "/api/v1/posts/2/"
      ...
    },
    ...
  ]
}
```

Pagination

- Pagination from the UI perspective
- REST API enables this
- Instagram et. al use REST API pagination for infinite scroll

...

Pagination

- List views should return a limited number of items
  - What if there were 10 million posts?
- Sensible default, e.g., 10 posts
  - `$ GET /api/v1/posts/ HTTP/1.0`
- Get the next 10 results
  - `$ GET /api/v1/posts/?page=2 HTTP/1.0`
- Customizable size
  - `$ GET /api/v1/posts/?size=20 HTTP/1.0`

Agenda

- REST API intro
- JSON
- Collections and pagination
- **REST API Verbs and Status codes.**
- Safety and idempotence

REST API verbs and status codes

- All our examples so far have been GET requests
  - GET request
  - 200 OK response

- This takes care of reading data. What about create, modify and delete?

REST API HTTP verbs

- GET: return a post
- POST: create a new post
- PATCH: modify part of an existing post
- PUT: replace the entire existing post
- DELETE: delete a post

POST request

- `POST` creates an object
- Request includes JSON body

```
POST /api/v1/posts/ HTTP/1.0
{
  "img_url": "122a7d27ca1d7420a1072f695d9290fad4501a41.jpg",
  "owner": "awdeorio",
  ...
}
```

POST response

- `POST` returns `201 CREATED` on success
- Response includes a copy of the created object
  - Object usually includes a link to itself

```
POST /api/v1/p/ HTTP/1.0
...
HTTP/1.0 201 CREATED
{
  "img_url": "122a7d27ca1d7420a1072f695d9290fad4501a41.jpg",
  "owner": "awdeorio",
  ...
  "url": "/api/v1/p/1/"
}
```

PATCH request

- `PATCH` modifies part of an existing object
- Request URL includes an ID
- Request includes JSON body

- Example: change the picture in a post
- Notice that the JSON body is short, and only contains the field that should be modified

```
PATCH /api/v1/p/1/ HTTP/1.0
{
  "img_url": "ad7790405c539894d25ab8dcf0b79eed3341e109.jpg",
}
```

PATCH response

- `PATCH` returns `200 OK` on success
- Response includes a copy of the **entire** modified object

```
PATCH /api/v1/p/1/ HTTP/1.0
...
HTTP/1.0 200 OK
{
  "img_url": "ad7790405c539894d25ab8dcf0b79eed3341e109.jpg",
  "owner": "awdeorio",
  ...
  "url": "/api/v1/p/1/"
}
```

PUT request

- `PUT` replaces an entire existing object
- Request URL includes an ID
- Request includes JSON body
- Example: replace an entire post
- The JSON body is long, and contains a replacement value for every field

```
PUT /api/v1/p/1/ HTTP/1.0
{
  "img_url":  ...,
  "owner": "jflinn",
  "owner_img_url": ...,
...
}
```

PUT response

- `PUT` returns `200 OK` on success
- Response includes a copy of the **entire** modified object

```
PUT /api/v1/p/1/ HTTP/1.0
...
HTTP/1.0 200 OK
{
  "img_url":  ...,
  "owner": "jflinn",
  "owner_img_url": ...,
...
}
```

DELETE request

- `DELETE` removes an object
- Request URL includes an ID
- No body in request

```
DELETE /api/v1/p/1/ HTTP/1.0
```

DELETE response on success

- `DELETE` returns `204 NO CONTENT` on success
- No body in response

```
DELETE /api/v1/p/1/ HTTP/1.0
...
HTTP/1.0 204 NO CONTENT
```

DELETE response on failure

- `GET` a deleted item, receive a 404 response

```
DELETE /api/v1/p/1/ HTTP/1.0
HTTP/1.0 204 NO CONTENT

GET /api/v1/p/1/ HTTP/1.0
HTTP/1.0 404 NOT FOUND
```

REST API status codes

- 200 OK
- 201 Created
  - Successful creation after POST
- 204 No Content
  - Successful DELETE
- 304 Not Modified
  - Used for conditional GET calls to reduce bandwidth usage
- 400 Bad Request
  - General error
- 401 Unauthorized
  - Missing or invalid authentication
- 403 Forbidden
  - User is not authorized
- 404 Not Found
  - Resource could not be found
- 409 Conflict
  - E.g., duplicate entries and deleting root objects when cascade-delete is not supported
- 500 Internal Server Error
  - General catch-all for server-side exceptions

http://www.restapitutorial.com/httpstatuscodes.html

Agenda

- REST API intro
- JSON
- Collections and pagination
- REST API Verbs and Status codes.
- **Safety and idempotence**

Safety and Idempotency

- *Safe*
  - Read-only request
  - Well-designed GET request
  - No change to state on server (the data on the server remains the same before and after the request)

- *Idempotent*
  - Sending the request **multiple times** leads to the same **result** on the server as sending it **once**
  - "If I ask this twice, will the same thing have happened as if I asked it once?"
  - Example: Deleting a file

Motivation for idempotency

- The Internet is a messy place – lots of network failures, network handovers, other people making requests

- When something is idempotent, we can retry the request

Motivation example

- You're on campus connected to Mwireless
- You click "like" on a post as you walk out the door
- App sends a request to the server to update the like count
- Phone loses MWireless connection before response arrives

- Q: What should the app do when it connects to the cellular network?
- A: If the request method is *idempotent* the app can send it again

Partner Q: idempotence in the real world

- Which of these human requests are **safe**? Which are **idempotent**?

- "What is your name?"
- "Turn the book to page 56."
- "What time is it?"
- "I would like a bowl with fried rice and orange chicken”
- "Turn left“
- "Turn East"

Partner Q: idempotence in HTTP

- Which of these HTTP verbs are safe?
  - GET
  - POST
  - PUT
  - DELETE
  - HEAD
- Which of them are idempotent?

GET, HEAD

- Safe
  - Doesn't change state on server
- Idempotent
  - Safe implies idempotent because no state is changing
  - GET and HEAD are read-only

PUT, POST

- PUT
  - Not safe: changes server state by modifying an object
  - Idempotent: a second duplicate PUT request won't have different effect than sending one
- POST
  - Not safe: changes server state by creating a new object
  - Not idempotent: two POST requests create two new objects

DELETE, PATCH

- DELETE
  - Not safe: changes state
  - Idempotent: A second DELETE means the object is still gone
- PATCH
  - It's complicated
  - With what HTTP spec says, we can't assume it's safe or that it's idempotent

Idempotent details

- Multiple identical requests should have the same effect *on the server* as a single request
- The same request can be made twice with no negative consequences *on the server*
- Does **not** mean that the same request always returns the same response
- **Does** mean that a request has NO side effects

- Why does Idempotency matter? If a request fails, can we automatically try again, but only if it is idempotent.

- Reference: HTTP RFC https://tools.ietf.org/html/rfc7231#section-4.2.2

Summary

- A client and a server can communicate via a REST API
- Two servers can communicate via a REST API
- REST APIs use HTTP
- REST APIs are machine-readable
- REST APIs usually return JSON data
- *Idempotent* methods can be repeated automatically

Public APIs

- GitHub
  https://developer.github.com/v3/
- LinkedIn
  https://developer.linkedin.com/
- Facebook
  https://developers.facebook.com/docs/graph-api
- Twitter
  https://dev.twitter.com/rest/public